

■ BOULDER DASH ■

Fiche Q&R; — Moteur de jeu • Processing (Java)

1. ARCHITECTURE GÉNÉRALE

Q : Quelle est la structure globale du programme ?

→ Le programme suit le pattern MVC allégé : BoulderDash.pde est le point d'entrée (setup/draw/keyPressed). Il délègue à GestionnaireJeu pour la logique, MoteurPhysique pour la physique, et Niveau pour la grille. Tous les fichiers .pde sont compilés en une seule classe Java par Processing.

Q : Comment la boucle de jeu fonctionne-t-elle ?

→ Processing appelle draw() à 60 FPS. À chaque frame, jeu.actualiser() met à jour les timers puis, si l'intervalle de 150 ms est écoulé (≈ 7 ticks/s), appelle physique.actualiser(). L'affichage reste à 60 FPS, la physique tourne à ~ 7 Hz.

```
if (t - dernierTick > intervalleTick) { physique.actualiser(niveau); dernierTick = t; }
```

Q : Pourquoi séparer la physique de l'affichage ?

→ La physique à 60 FPS serait trop rapide (rochers/ennemis impossibles à suivre). Séparer les deux permet un rendu fluide (60 FPS) tout en maintenant une vitesse de jeu jouable (~ 7 ticks/s) indépendamment du matériel.

Q : Quel est le rôle de la classe Element ?

→ Element est la classe abstraite dont héritent tous les objets du jeu : Rocher, Diamant, Joueur, Ennemi, Mur, MurMagique, Sortie, Terre. Elle stocke x, y et déclare les méthodes afficher() et éventuellement actualiser().

2. GRILLE ET DÉPLACEMENTS

Q : Comment la grille est-elle représentée en mémoire ?

→ La grille est un tableau 2D Element[][] de dimensions cols × rows. Chaque case contient soit null (vide), soit une référence à un objet Element. Les bords hors limites retournent un Mur virtuel (getElement()).

```
Element[][] grille; // null = case vide
```

Q : Comment fonctionne deplacerElement() ?

→ La méthode efface la case d'origine (grille[e.x][e.y] = null), met à jour les coordonnées de l'objet (e.x = nx, e.y = ny), puis écrit la référence dans la nouvelle case. Aucune copie d'objet : c'est toujours la même instance qui se déplace.

Q : Comment le fichier de niveau est-il chargé ?

→ loadStrings('levelN.txt') lit le fichier texte. La première ligne peut contenir les métadonnées (NORMAL:10 ou BONUS:5). Chaque caractère est converti en objet : W=Mur, O=Rocher, *=Diamant, P=Joueur1, Q=Joueur2 (multi seulement), X=Ennemi, E=Sortie, .=Terre.

Q : Pourquoi Q est-il ignoré en mode solo ?

→ Le constructeur Niveau(index, multi) reçoit un booléen avecJoueur2. En solo ce booléen vaut false : le 'Q' est simplement ignoré (case vide). Cela permet de conserver un unique jeu de fichiers pour les deux modes.

```
if (c == 'Q') { if (avecJoueur2) { joueur2 = new Joueur(x, y, 2); ... } }
```

3. PHYSIQUE — GRAVITÉ ET CHUTE

Q : Comment la gravité est-elle simulée ?

→ MoteurPhysique parcourt la grille de bas en haut ($y = \text{rows}-2 \rightarrow 0$). Pour chaque Rocher ou Diamant, si la case en dessous est vide, l'objet est déplacé d'une case vers le bas et son flag enChute passe à true.

Q : Pourquoi parcourir de bas en haut ?

→ Si on parcourt de haut en bas, un rocher déplacé vers le bas serait immédiatement revisité dans le même tick et tomberait de plusieurs cases à la fois. Le parcours bas→haut garantit qu'un rocher ne tombe que d'une case par tick.

Q : Comment les rochers glissent-ils latéralement ?

→ Quand un rocher ou diamant repose sur un autre objet arrondi (Rocher ou Diamant), MoteurPhysique tente de le faire glisser à gauche si les deux cases (côté + dessous-côté) sont libres, sinon à droite. Cela crée l'effet d'avalanche caractéristique.

```
if (getElement(x-1,y)==null && getElement(x-1,y+1)==null) deplacerElement(e, x-1, y);
```

Q : À quelle condition un rocher écrase-t-il le joueur ?

→ Deux conditions doivent être réunies : (1) le rocher était déjà en chute au tick précédent ($\text{enChute} == \text{true}$) ET (2) le joueur n'est pas invulnérable ($\text{millis}() \geq \text{joueur.bloqueJusquA}$). Un rocher posé sur le joueur ne le tue pas.

```
if (enChuteActuellement && millis() >= jEcrase.bloqueJusquA) jEcrase.vivant = false;
```

Q : Qu'est-ce que le flag aBoge[] ?

→ C'est un tableau booléen réinitialisé à chaque tick. Il empêche qu'un élément soit déplacé deux fois dans le même tick (ex: un rocher glisse puis retombe sur la case libérée). Dès qu'un élément bouge, sa case de destination est marquée.

4. MACHINE À ÉTATS

Q : Quels sont les états du jeu et leur rôle ?

→ ETAT_JEU (0) : partie en cours. ETAT_GAMEOVER (1) : fin sans victoire. ETAT_VICTOIRE (2) : tous les niveaux terminés (solo). ETAT_PAUSE (3) : jeu gelé. ETAT_SAISIE_PSEUDO (4) : entrée du nom pour le top 5. ETAT_FIN_MULTI (5) : fin de partie multijoueur. ETAT_TRANSITION (6) : écran inter-niveaux (2,5 s).

Q : Comment la pause gère-t-elle le timer sans tricher ?

→ À la mise en pause, `pauseDebut = millis()` est sauvegardé. À la reprise, la durée de pause est ajoutée aux deux références temporelles (`dernierTempsMillis` et `dernierTick`) pour que les timers repartent exactement où ils s'étaient arrêtés.

```
int duree = millis() - pauseDebut; dernierTempsMillis += duree; dernierTick += duree;
```

Q : Comment l'écran de transition inter-niveaux fonctionne-t-il ?

→ Quand tous les diamants sont collectés, `etat` passe à `ETAT_TRANSITION` et `tempsTransition = millis() + 2500`. À chaque frame, `actualiser()` vérifie `millis() >= tempsTransition` ; quand c'est vrai, il charge le niveau suivant. Le rendu affiche un overlay semi-transparent avec les scores et un compte à rebours.

5. JOUEUR ET DÉPLACEMENTS

Q : Comment le joueur se déplace-t-il ?

→ `keyPressed()` appelle `jeu.gererEntree(key, keyCode)` qui appelle `joueur.deplacer()`. La méthode calcule `dx/dy` selon la touche, récupère la case cible via `getElement()`, et l'action dépend du type : `null/Terre` → déplacement libre, `Diamant` → collecte, `Rocher` → pousse si la case derrière est libre, `Sortie` → niveau suivant si ouverte.

Q : Pourquoi les touches sont-elles différentes en solo et en multi ?

→ En multi, les flèches sont réservées au Joueur 2. Si J1 les utilisait aussi, les deux joueurs bougeraient en même temps sur la même touche. Le code vérifie `jeu.modeMulti` pour choisir le mapping : `ZQSD` seul en multi, `ZQSD + flèches` en solo.

Q : Comment fonctionne la poussée d'un rocher ?

→ Le joueur ne peut pousser un rocher que horizontalement (`dy == 0`). Le rocher est poussé seulement si la case derrière lui (`px2 = nx + dx`) est nulle. Le rocher est d'abord déplacé, puis le joueur prend sa place.

```
if (cible instanceof Rocher && dy == 0) { if (getElement(px2, ny) == null) {  
    deplacerElement(cible, px2, ny); deplacerElement(this, nx, ny); } }
```

6. MULTIJOUEUR — RESPAWN ET INVULNÉRABILITÉ

Q : Que se passe-t-il quand un joueur meurt en mode multi ?

→ À la fin du tick physique, `GestionnaireJeu` vérifie `joueur.vivant`. Si `false`, il appelle `respawnerJoueur()` : le joueur est retiré de sa case actuelle, repositionné à `startX/startY`, `vivant = true`, et `bloqueJusquA = millis() + 3000`. L'autre joueur continue sans interruption, la carte n'est pas rechargée.

Q : Comment l'invulnérabilité post-respawn est-elle implémentée ?

→ Le champ `bloqueJusquA` stocke un timestamp futur (`millis() + 3000`). Toutes les sources de dégâts vérifient `millis() >= bloqueJusquA` avant de tuer. Le même champ bloque aussi le déplacement dans `deplacer()`. Valeur par défaut = 0 (toujours dans le passé → jamais bloqué en temps normal).

```
if (millis() < bloqueJusquA) return; // dans deplacer() if (enChuteActuellement  
&& millis() >= jEcrase.bloqueJusquA) ... // dans MoteurPhysique
```

Q : Que se passe-t-il si la case de départ est occupée lors du respawn ?

→ `resawnerJoueur()` vérifie `grille[j.x][j.y] == j` avant de libérer la case actuelle. Si un rocher a écrasé le joueur et occupe sa case, la case n'est PAS effacée. Le joueur est quand même placé à `startX/startY`, ce qui peut provoquer une superposition temporaire résolue au tick suivant.

Q : Comment le clignotement du joueur bloqué est-il rendu ?

→ Dans `afficher()`, si `bloqueJusquA > millis()`, la variable `bloque` est vraie. Si `frameCount % 14 < 7` (la moitié des frames), seul le fond noir est dessiné et la méthode retourne immédiatement, rendant le joueur invisible une frame sur deux.

```
if (bloque && frameCount % 14 < 7) { fill(30); rect(...); return; }
```

7. SCORES ET AVANCEMENT DE NIVEAU

Q : Comment le score est-il calculé ?

→ Chaque diamant collecté vaut 10 points (`points × 10` dans `ajouterScore`). En solo le score est global (`scoreGlobal`). En multi chaque joueur a son propre score (`scoreJ1`, `scoreJ2`) et son compteur de diamants (`diamantsJ1`, `diamantsJ2`).

Q : Pourquoi utiliser le flag `avancerNiveauAuto` plutôt d'appeler `niveauSuivant()` directement ?

→ `ajouterScore()` est appelé depuis `Joueur.deplacer()`, qui est au milieu d'un tick physique. Appeler `niveauSuivant()` à ce moment réinitialiserait la grille pendant son parcours, corrompant les itérateurs. Le flag reporte le changement de niveau au début du prochain appel de `actualiser()`, où la grille est dans un état stable.

Q : En solo, à quelle condition la sortie s'ouvre-t-elle ?

→ Quand `diamantsRecoltes >= scoreCible` (`nbDiamantsRequis` du niveau). La sortie n'est pas la mort de tous les diamants : seul le quota requis suffit. En mode BONUS, la sortie est ouverte dès le départ.

Q : En multi, comment le passage de niveau est-il déclenché ?

→ `compterDiamantsRestants()` parcourt toute la grille et compte les instances `Diamant`. Dès que ce total atteint 0 (tous les diamants ont été collectés, pas seulement le quota), `avancerNiveauAuto` passe à `true` et la transition commence.

8. ENNEMIS ET MUR MAGIQUE

Q : Comment se déplace l'ennemi ?

→ L'ennemi se déplace horizontalement (`dx = +1` ou `-1`). À chaque tick physique, il tente d'avancer d'une case. Si la case est occupée (mur, rocher...), il fait demi-tour immédiatement et tente d'avancer dans l'autre direction dans le même tick.

Q : Comment l'ennemi tue-t-il un joueur ?

→ Si la case cible de l'ennemi est un `Joueur` (`instanceof Joueur`), et que le joueur n'est pas invulnérable (`millis() >= bloqueJusquA`), `joueur.vivant = false`. Après son déplacement, l'ennemi vérifie aussi si un joueur occupe sa nouvelle case (cas où le joueur marche vers l'ennemi).

Q : Comment un rocher peut-il tuer un ennemi ?

→ Dans MoteurPhysique, si la case sous le rocher contient un Ennemi et que le rocher était en chute libre (enChute == true), l'ennemi est retiré de la liste ennemis, le rocher prend sa place, et un bonus de 100 points est ajouté au score.

Q : Que fait le mur magique ?

→ Un Rocher en chute libre (enChute == true) traversant un MurMagique est supprimé, et un Diamant apparaît sous le mur (à y+2) — à condition que la case sous le mur soit libre ou de la Terre. Un diamant qui tombe n'active pas le mur magique.

```
// Rocher → traverse MurMagique → Diamant apparaît en y+2 niveau.grille[bx][by]  
= new Diamant(bx, by);
```